Milestone 5 Report

Smart MCQ Solver - Model Ensembling with DeBERTa and RoBERTa

A fully explained walkthrough of the milestone-5.ipynb notebook: every question, the reasoning behind each approach, the exact code used, the notebook's output, and the final answer, written so that anyone unfamiliar with the notebook can understand what was done and why.

Source notebook: milestone-5.ipynb **Training script:** src/mcq_deberta_roberta.py **Dataset:** Kaggle "smart-mcq-solver-challenge"

---

1. Project Overview

This notebook introduces **model ensembling** for the same multiple-choice question problem tackled in Milestones 1 to 4. Each row provides a question (prompt) and five candidate options (A through E), and the goal is to identify the correct option (answer).

Milestones 1 to 4 asked how well a single model could do. This milestone asks a different question: given two models that were each fine-tuned separately, can combining their predictions beat either one alone? The notebook is organised as a guided sequence of 10 questions covering independent inference with softmax, simple probability averaging, weighted averaging, Top-3 ranking in Kaggle format, a full test-set run, test-time augmentation, and four measurements of what the ensemble actually changed compared with a single model.

The key design decision is that ensembling happens at the **probability** level, not the logit level. Each model produces raw logits on its own scale, which are not comparable across models. Applying softmax first converts both to probability distributions over the same five classes, which can then be averaged meaningfully.

1.1 Environment and Setup

The notebook begins by verifying the software and hardware environment, then confirms the three competition data files are present before any analysis begins.

item	value
Python	3.10.12
Platform	Linux 6.6.114.1-microsoft-standard-WSL2 (WSL2)
NumPy	2.2.6
Pandas	2.3.3
PyTorch	2.5.1+cu121
Transformers	4.52.4 (5.13.0 after the mid-notebook upgrade)
Device	CUDA (NVIDIA GeForce RTX 3050 Laptop GPU, 4.29 GB VRAM)
Model 1	DeBERTa-v3-small, fine-tuned, loaded from models/deberta-mcq
Model 2	RoBERTa-base, fine-tuned, loaded from models/roberta-mcq
Task head	AutoModelForSequenceClassification, num_labels = 5

Note the 4.29 GB VRAM. This is the constraint that shapes the whole milestone: two transformers must be resident at once, so both are loaded in evaluation mode with gradients disabled and inference runs in batches of 16.

1.2 Dataset

The dataset is read from three CSV files with pandas. Inspection with .shape and .isna().sum() confirms the data is clean before any modelling begins.

item	value
Files	train.csv, test.csv, sample_submission.csv
train.csv shape	2000 rows x 8 columns
test.csv shape	500 rows x 7 columns
sample_submission.csv shape	500 rows x 2 columns
Columns	id, prompt, A, B, C, D, E, answer (test.csv omits answer)
Missing values	None, all columns fully populated

1.3 How the Two Models Were Trained

Both models come from src/mcq_deberta_roberta.py and were trained identically, which is what makes comparing them fair:

setting	value
task framing	5-way sequence classification (labels A=0 to E=4)
epochs	3
learning rate	2e-5
batch size	8
max length	512 tokens
split	90 / 10, random_state=42, stratified on the answer label

Each question is flattened into **one** input string containing the prompt and all five options together:

```
Question: <prompt>
A) <option A>
B) <option B>
C) <option C>
D) <option D>
E) <option E>
```

The model reads that single string and outputs five logits, one per letter. This is worth contrasting with the main project notebook, which instead scores each (prompt, option) pair separately with a multiple-choice head. The approach here is simpler and lets a standard classification head be used, but it makes the prediction depend on the order the options appear in, because the letters are baked into the text the model reads.

1.4 Shared Helper Functions

Three helpers are defined once and reused by every question.

get_probs(model, tokenizer, texts) runs batched inference under torch.no_grad(), applies softmax to the logits, and returns an array of shape (n_rows, 5). The softmax is the important part: it is what puts both models on the same scale so their outputs can be combined.

top3_string(prob_row) sorts one probability row descending and joins the top three letters with spaces, producing the exact format Kaggle expects, for example “E C A”.

mapk_score(true_labels, pred_top3_lists, k=3) implements MAP@3 by hand: 1.0 if the correct letter is ranked first, 0.5 if second, 1/3 if third, 0 otherwise, averaged over all rows.

1.5 Recreating the Validation Split

Before any question is answered, the notebook recreates the exact split the training script used, with the same test_size=0.1, random_state=42 and stratification. This matters: if the split were regenerated with a different seed, validation rows would leak into what the models were trained on, and every score afterwards would be meaningless.

---

2. Table of Contents

Independent inference and simple ensembling - Q1. Highest-probability option from DeBERTa alone - Q2. Highest-probability option after simple averaging - Q3. Highest-probability option after weighted averaging

Producing a submission - Q4. Top-3 prediction string for row 25 - Q5. Row count in the generated submission.csv

Test-time augmentation - Q6. Top-1 changes after instruction-prefix augmentation

Measuring what the ensemble actually changed - Q7. Top-1 disagreements between DeBERTa and the weighted ensemble - Q8. Rows with a positive confidence gain - Q9. Rows with any change in Top-3 ordering

Final evaluation - Q10. MAP@3 of the weighted ensemble on validation

---

3. Independent Inference and Simple Ensembling

Question 1: Which option receives the highest probability from DeBERTa, and what is that probability?

Approach and explanation. Before combining anything, each model must be run on its own so there is a reference point. Row index 25 is formatted into the single-string layout described in 1.3, passed through DeBERTa, and softmaxed into five probabilities. np.argmax finds the highest.

Code

```
row25 = df.iloc[[25]]
text25 = [build_text(row25.iloc[0])]

p_deberta_25 = get_probs(deberta_model, deberta_tok, text25)[0]
p_roberta_25 = get_probs(roberta_model, roberta_tok, text25)[0]

best_d = np.argmax(p_deberta_25)
print("Q1 - DeBERTa top prediction:", ID2LABEL[best_d],
      "prob =", round(float(p_deberta_25[best_d]), 4))
```

Notebook output

```
Q1 - DeBERTa top prediction: E prob = 0.9969
```

Answer: E, with probability 0.9969.

A probability of 0.9969 is extremely high. The model is not weighing options against each other, it is nearly certain. That is worth noticing early, because it limits how much any ensemble can change: a second model would need to be both confident and disagree in order to move the result.

Question 2: Which option receives the highest averaged probability after simple probability ensembling?

Approach and explanation. Simple ensembling gives both models an equal vote:

```
P(average) = [P(DeBERTa) + P(RoBERTa)] / 2
```

Because both arrays are already probability distributions over the same five classes, the average is itself a valid distribution. No calibration is needed.

Code

```
avg_probs_25 = (p_deberta_25 + p_roberta_25) / 2
best_avg = np.argmax(avg_probs_25)
print("Q2 - Simple-avg top prediction:", ID2LABEL[best_avg],
      "prob =", round(float(avg_probs_25[best_avg]), 4))
```

Notebook output

This cell’s output was not saved in the notebook, so it should be re-run before submission. The answer is nonetheless

recoverable from the two figures that were printed. Q3 reports the weighted result as 0.9687, and the weighting is known, so RoBERTa's probability for E follows:

```
P_roberta(E) = (0.9687 - 0.7 x 0.9969) / 0.3 = 0.9029
P(average)   = (0.9969 + 0.9029) / 2         = 0.9499
```

Answer: E, with probability approximately 0.9499.

Both models independently favour E, so averaging cannot change the winner. It only lowers the confidence, from 0.9969 to 0.9499, because RoBERTa is the less certain of the two.

Question 3: Which option is ranked first after weighted ensembling?

Approach and explanation. Equal weighting assumes both models are equally trustworthy. Weighted ensembling relaxes that, giving DeBERTa 0.70 and RoBERTa 0.30 because DeBERTa is the stronger model:

```
P(final) = 0.7 x P(DeBERTa) + 0.3 x P(RoBERTa)
```

The weights sum to 1, so the result remains a valid probability distribution.

Code

```
w_deberta, w_roberta = 0.70, 0.30
weighted_probs_25 = w_deberta * p_deberta_25 + w_roberta * p_roberta_25
best_w = np.argmax(weighted_probs_25)
print("Q3 - Weighted-avg top prediction:", ID2LABEL[best_w],
      "prob =", round(float(weighted_probs_25[best_w]), 4))
```

Notebook output

```
Q3 - Weighted-avg top prediction: E prob = 0.9687
```

Answer: E, with probability 0.9687.

The three results so far line up as expected: DeBERTa alone 0.9969, weighted 0.9687, simple average 0.9499. The more weight RoBERTa receives, the lower the confidence, because RoBERTa is less certain about this row. The predicted letter never changes.

---

4. Producing a Submission

Question 4: What is the Top-3 prediction string for row index 25?

Approach and explanation. Kaggle scores by MAP@3, so a submission gives three ranked letters rather than one. Partial credit is available: 1.0 for a correct first guess, 0.5 for second, 1/3 for third. Submitting only the top letter would discard that.

Code

```
q4_top3 = top3_string(weighted_probs_25)
print("Q4 - Top-3 for row 25:", q4_top3)
```

Notebook output

```
Q4 - Top-3 for row 25: E C A
```

Answer: "E C A".

E carries 0.9687 of the probability mass, so C and A are being ranked from the remaining 0.031. Those two positions are nearly arbitrary here, but they cost nothing and can only add score.

Question 5: How many prediction rows are in the generated submission.csv?

Approach and explanation. The pipeline validated on one row is now applied to all 500 test rows in one pass: build

the text for every row, run both models, combine at 0.70 / 0.30, take the Top-3 of each, and write a two-column CSV.

Code

```
test_df = pd.read_csv(f"{DATA_DIR}/test.csv")
test_texts = test_df.apply(build_text, axis=1).tolist()

p_deberta_test = get_probs(deberta_model, deberta_tok, test_texts)
p_roberta_test = get_probs(roberta_model, roberta_tok, test_texts)
weighted_test = w_deberta * p_deberta_test + w_roberta * p_roberta_test

top3_preds_test = [top3_string(row) for row in weighted_test]

submission = pd.DataFrame({"id": test_df["id"], "prediction": top3_preds_test})
submission.to_csv(f"{REPO_ROOT}/src/submission.csv", index=False)

print("Q5 - Number of prediction rows in submission.csv:", len(submission))
```

Notebook output

```
Q5 - Number of prediction rows in submission.csv: 500
```

Answer: 500 rows, excluding the header.

This matches test.csv exactly, which is the check that matters: a submission with a different row count is rejected regardless of how good the predictions are.

---

5. Test-Time Augmentation

Question 6: How many of the first 50 rows produce a different Top-1 after TTA?

Approach and explanation. Test-time augmentation asks the same question in more than one way and averages the answers, on the theory that a model sensitive to phrasing will be steadied by it. Here each of the first 50 prompts is run twice through DeBERTa: once as-is, and once with the prefix “Answer the following multiple-choice question carefully:”. The two probability arrays are averaged and the Top-1 compared against the original.

Code

```
first50 = test_df.iloc[:50].reset_index(drop=True)

texts_orig = [build_text(r) for _, r in first50.iterrows()]
texts_aug = [build_text(r, prefix="Answer the following multiple-choice question carefully: ")
             for _, r in first50.iterrows()]

p_orig = get_probs(deberta_model, deberta_tok, texts_orig)
p_aug = get_probs(deberta_model, deberta_tok, texts_aug)
p_tta = (p_orig + p_aug) / 2

top1_orig = np.argmax(p_orig, axis=1)
top1_tta = np.argmax(p_tta, axis=1)

q6_diff_count = int(np.sum(top1_orig != top1_tta))
print("Q6 - Rows with different Top-1 after TTA (first 50):", q6_diff_count)
```

Notebook output

```
Q6 - Rows with different Top-1 after TTA (first 50): 0
```

Answer: 0 rows.

Not one prediction changed. The honest reading is that the model is insensitive to this particular prefix, which is a reasonable outcome given how confident it is: shifting a prediction sitting at 0.99 requires a large disagreement, and a short instruction prefix does not supply one. A zero here is a real result, not a failure, but it does mean TTA of this kind adds nothing on this dataset.

---

6. Measuring What the Ensemble Actually Changed

The next three questions all ask the same underlying thing from different angles: did adding RoBERTa make any practical difference?

Question 7: How many of the first 100 rows have different Top-1 predictions between DeBERTa alone and the weighted ensemble?

Approach and explanation. For each of the first 100 test rows, take DeBERTa's Top-1 and the weighted ensemble's Top-1 and count the disagreements.

Code

```
first100 = test_df.iloc[:100].reset_index(drop=True)
texts_100 = [build_text(r) for _, r in first100.iterrows()]

p_deberta_100 = get_probs(deberta_model, deberta_tok, texts_100)
p_roberta_100 = get_probs(roberta_model, roberta_tok, texts_100)
p_weighted_100 = w_deberta * p_deberta_100 + w_roberta * p_roberta_100

top1_deberta_100 = np.argmax(p_deberta_100, axis=1)
top1_weighted_100 = np.argmax(p_weighted_100, axis=1)

q7_diff_count = int(np.sum(top1_deberta_100 != top1_weighted_100))
print("Q7 - Rows with different Top-1 (DeBERTa vs Weighted Ensemble, first 100):",
      q7_diff_count)
```

Notebook output

```
Q7 - Rows with different Top-1 (DeBERTa vs Weighted Ensemble, first 100): 0
```

Answer: 0 rows.

At a 0.70 weight, DeBERTa needs only to exceed 0.5 confidence to be mathematically unbeatable by RoBERTa on that row. Given how often DeBERTa sits above 0.99, the ensemble's Top-1 is effectively DeBERTa's Top-1 by construction.

Question 8: How many of the first 100 rows have a positive confidence gain?

Approach and explanation. Confidence gain is the ensemble's top probability minus DeBERTa's top probability. A positive value would mean blending made the model more certain.

Code

```
conf_deberta_100 = p_deberta_100[np.arange(len(p_deberta_100)), top1_deberta_100]
conf_weighted_100 = p_weighted_100[np.arange(len(p_weighted_100)), top1_weighted_100]

confidence_gain = conf_weighted_100 - conf_deberta_100
q8_positive_count = int(np.sum(confidence_gain > 0))
print("Q8 - Rows with positive confidence gain (first 100):", q8_positive_count)
```

Notebook output

```
Q8 - Rows with positive confidence gain (first 100): 0
```

Answer: 0 rows.

This follows arithmetically from Q7 rather than being an independent finding. Since the winning letter is always DeBERTa's, the blended probability is `0.7 x P_deberta + 0.3 x P_roberta` for that letter, which can only exceed `P_deberta` if RoBERTa is *more* confident about the same letter. Because DeBERTa is consistently the more confident model, the gain is negative on every row. Ensembling here reduces confidence rather than raising it.

Question 9: How many of the first 100 rows have any change in their Top-3 ordering?

Approach and explanation. Q7 and Q8 both returned zero, which could suggest RoBERTa is contributing nothing at all. This question tests that properly, by comparing the full three-letter ranking rather than only the winner.

Code

```
top3_deberta_100 = [top3_string(row) for row in p_deberta_100]
top3_weighted_100 = [top3_string(row) for row in p_weighted_100]

q9_diff_count = sum(1 for a, b in zip(top3_deberta_100, top3_weighted_100) if a != b)
print("Q9 - Rows with any change in Top-3 ranking (first 100):", q9_diff_count)
```

Notebook output

```
Q9 - Rows with any change in Top-3 ranking (first 100): 58
```

Answer: 58 rows.

This is the most informative result in the milestone. RoBERTa cannot overturn DeBERTa's first choice, but it reorders positions 2 and 3 on **58 of 100 rows**. That is where its contribution lives.

It also matters for the metric. Under MAP@3, moving the correct letter from rank 3 to rank 2 is worth 0.5 minus 1/3, about 0.167 for that row. So an ensemble that never changes a single Top-1 prediction can still change the score, and Q7 and Q8 alone would have led to the wrong conclusion that ensembling was pointless here.

---

7. Final Evaluation

Question 10: What is the MAP@3 score of the weighted ensemble on the first 100 validation samples?

Approach and explanation. The final step scores the pipeline against known answers using the held-out validation rows from the split recreated in 1.5.

Code

```
val_100 = val_df.iloc[:100].reset_index(drop=True)
val_texts_100 = [build_text(r) for _, r in val_100.iterrows()]

p_deberta_val = get_probs(deberta_model, deberta_tok, val_texts_100)
p_roberta_val = get_probs(roberta_model, roberta_tok, val_texts_100)
p_weighted_val = w_deberta * p_deberta_val + w_roberta * p_roberta_val

top3_val_lists = [[ID2LABEL[i] for i in np.argsort(-row)[:3]] for row in p_weighted_val]
true_labels_val = val_100["answer"].tolist()

map3 = mapk_score(true_labels_val, top3_val_lists, k=3)
print("Q10 - MAP@3 score (first 100 validation samples):", round(float(map3), 4))
```

Notebook output

```
Q10 - MAP@3 score (first 100 validation samples): 1.0
```

Answer: MAP@3 = 1.0.

This number should not be reported without its caveat. A perfect 1.0 means the correct answer was ranked first on all 100 rows, which is not a plausible level of reasoning ability on unseen questions. The cause is a property of the dataset that was diagnosed in the main project work: train.csv contains many near-duplicate questions, roughly 2,000 rows collapsing to about 613 distinct questions once questions differing only in their instruction wording are grouped together.

A stratified random split, as used here, therefore places near-identical copies of the same question on both sides of the boundary. The models saw those questions during training and are recalling them at validation time rather than solving them. The main project notebook addresses this by clustering near-duplicates and assigning whole clusters to one side, which reduces cross-boundary duplication from about 79.5% to 0.0% and brings scores down to a believable range.

For this milestone the 1.0 is the correct answer to the question as posed, since the question specifies this split. It is reported here with the explanation attached rather than presented as evidence the ensemble is perfect.

---

8. Summary of Answers

question	answer
Q1. DeBERTa top prediction, row 25	E, probability 0.9969
Q2. Simple-average top prediction	E, probability approximately 0.9499 (derived, cell output not saved)
Q3. Weighted-average top prediction	E, probability 0.9687
Q4. Top-3 string for row 25	E C A
Q5. Rows in submission.csv	500
Q6. Top-1 changes after TTA, first 50	0
Q7. Top-1 disagreements, first 100	0
Q8. Rows with positive confidence gain	0
Q9. Rows with Top-3 reordering	58
Q10. MAP@3, first 100 validation rows	1.0 (inflated by near-duplicate leakage)

9. What This Milestone Demonstrates

Ensembling must happen after softmax. Logits from two separately fine-tuned models live on different scales and cannot be averaged. Softmax puts both on a common probability scale first, which is what makes the arithmetic in Q2 and Q3 valid.

A dominant weight makes the ensemble mostly decorative for Top-1. At 0.70 / 0.30, any row where DeBERTa exceeds 0.5 confidence is decided before RoBERTa is consulted. Q7 and Q8 returning zero is the arithmetic consequence, not a coincidence.

Where the ensemble does contribute is below rank 1. Q9’s 58 of 100 shows RoBERTa reordering the second and third positions on most rows. Under a metric that awards partial credit, that is worth measuring, and it is the finding the milestone is built to surface.

A perfect score is a signal to investigate, not to celebrate. Q10’s 1.0 traces directly to near-duplicate questions straddling a random split. Recognising that, and saying so, is more useful than reporting the number on its own.

Two open items for a future iteration. The Q2 cell should be re-run so its output is captured rather than derived. And a weight sweep would be worthwhile: 0.70 / 0.30 was chosen because DeBERTa is the stronger model, but it was never tuned, and a more balanced weighting is the only way RoBERTa could influence Top-1 at all.